

Source Code of App: base

apps.py

```
from django.apps import AppConfig
```

```
class BaseConfig(AppConfig):
```

```
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'base'
```

form.py

```
from django import forms
from django.contrib.auth.forms import PasswordChangeForm
from django.contrib.auth.forms import UserCreationForm
```

```
from base.models import UserInfomation , PersonalInformation
from django.contrib.auth.forms import UserCreationForm
```

```
class UserRegisterForms(UserCreationForm):
```

```
    email = forms.EmailField(required=True)
    contact_number = forms.CharField(max_length=20, required=True)
    class Meta:
        model = UserInfomation
        fields = ['username', 'email', 'first_name', 'last_name', 'contact_number', 'password1', 'password2']
```

```
    def save(self , commit=True):
        user = super().save(commit=False)
        user.email = self.cleaned_data['email']
        user.contact_number = self.cleaned_data['contact_number']
        user.role = 'Beginner Sitter'
        if commit:
            user.save()
        return user
```

```
class UserInfomationForm(forms.ModelForm):
```

```
    class Meta:
        model = UserInfomation
        fields = ['username', 'email', 'contact_number', 'role']
        widgets = {
            'role': forms.TextInput(attrs={'readonly': 'readonly'})
        }
```

```
class PersonalInformationForm(forms.ModelForm):
```

```
    class Meta:
        model = PersonalInformation
        fields = ['goal', 'job_name', 'job_type', 'job_hours', 'break_hours','age', 'height', 'weight', 'has_pain']
```

```
class UserChangeForm(forms.ModelForm):
```

```
    class Meta:
        model = UserInfomation
        fields = ['username']
        widgets = {
            'username': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'กรอกชื่อผู้ใช้งานใหม่'}),
        }
```

```
class CustomPasswordChangeForm(PasswordChangeForm):
```

```
    old_password = forms.CharField(
        label="รหัสผ่านเดิม",
        widget=forms.PasswordInput(attrs={'class': 'form-control'})
    )
    new_password1 = forms.CharField(
        label="รหัสผ่านใหม่",
        widget=forms.PasswordInput(attrs={'class': 'form-control'})
    )
    new_password2 = forms.CharField(
        label="ยืนยันรหัสผ่านใหม่",
        widget=forms.PasswordInput(attrs={'class': 'form-control'})
    )
```

```
class ContactForm(forms.Form):
```

```
    name = forms.CharField(max_length=100, required=True)
    email = forms.EmailField(required=True)
    message = forms.CharField(widget=forms.Textarea, required=True)
```

models.py

```
from django.db import models
```

```

from django.contrib.auth.models import AbstractUser
from django.core.exceptions import ValidationError
from django.utils import timezone

class UserInfomation(AbstractUser):
    contact_number = models.CharField(max_length= 20 , blank = True)
    role = models.CharField(max_length= 50 )
    created_at = models.DateTimeField(auto_now_add= True)

    def __str__(self):
        return self.username

class PersonalInformation(models.Model):
    user = models.OneToOneField(UserInfomation, on_delete=models.CASCADE, null=True, blank=True)
    goal = models.CharField(
        max_length=50,
        choices=[
            ('Improve posture', 'Improve posture'),
            ('Health maintenance', 'Health maintenance'),
            ('Posture correction', 'Posture correction')
        ],
        null=True,
        blank=True
    )
    job_name = models.CharField(max_length=100, null=True, blank=True)
    job_type = models.CharField(max_length=100, null=True, blank=True)
    job_hours = models.FloatField(null=True, blank=True)
    break_hours = models.FloatField(null=True, blank=True)
    age = models.IntegerField(null=True, blank=True)
    height = models.FloatField(null=True, blank=True)
    weight = models.FloatField(null=True, blank=True)
    has_pain = models.BooleanField(default=False, null=True, blank=True)

    def __str__(self):
        return f'{self.user.username if self.user else "No User"} - {self.goal or "No Goal"}'

class PostureDetection(models.Model):
    user = models.ForeignKey(UserInfomation, on_delete=models.CASCADE)
    timestamp = models.DateTimeField(default=timezone.now, null=True) # ย้าย timestamp มาที่นี่
    score = models.IntegerField()

    def __str__(self):
        return f'{self.user.username} - Posture Score: {self.score} - Timestamp: {self.timestamp}'

class UserUsageHistory(models.Model):
    posture_detection = models.ForeignKey(PostureDetection, on_delete=models.CASCADE)
    detect_type = models.CharField(
        max_length=50,
        choices=[
            ('Photo Detection', 'Photo Detection'),
            ('Side-part Detection', 'Side-part Detection')
        ]
    )
    detection_time = models.FloatField(null=True, blank=True) # ย้าย detection_time มาที่นี่

    def clean(self):
        # ถ้า detect_type เป็น 'Photo Detection' ต้องไม่ให้กรอกค่า detection_time
        if self.detect_type == 'Photo Detection' and self.detection_time is not None:
            raise ValidationError({'detection_time': 'Detection time must be empty for Photo Detection.'})

    def __str__(self):
        return f'{self.posture_detection.user.username} - {self.detect_type} - Detection Time: {self.detection_time}'

class NotificationLog(models.Model):
    user = models.ForeignKey(UserInfomation, on_delete=models.CASCADE)
    message = models.TextField()
    notification_time = models.DateTimeField(auto_now_add=True)
    admin_message = models.TextField(blank=True)

    def __str__(self):
        return f'Notification for {self.user.username} at {self.notification_time}'

```

```

urls.py
from django.contrib import admin
from django.contrib.auth import views as auth_views
from django.contrib.auth.views import LogoutView

from django.urls import path , include
from base.views import *
app_name = 'base'
urlpatterns = [
    path('', HomepageView.as_view() , name='home' ),

    path('signup/', UserRegisterview.as_view(), name='Register'),
    path('login/', Userloginview.as_view() , name='Login'),
    path('logout/', auth_views.LogoutView.as_view(next_page=reverse_lazy('base:Login')), name='logout'),

    path('password-reset/', ResetPasswordview.as_view(), name='password_reset'),

    path('password-reset/done/', auth_views.PasswordResetDoneView.as_view(), name='password_reset_done'),

    path('password-reset-confirm///',
        auth_views.PasswordResetConfirmView.as_view(template_name='password_reset_confirm.html',
                                                    success_url=reverse_lazy('base:password_reset_complete')),
        name='password_reset_confirm'),

    path('password-reset-complete/',
        auth_views.PasswordResetCompleteView.as_view(template_name='password_reset_complete.html'),
        name='password_reset_complete'),

    path('update-personal-information/', PersonalInformationUpdateView.as_view(), name='update_personal_information'),

    path('change_username/', ChangeUsernameView.as_view(), name='change_name'),
    path('change_password/', ChangePasswordView.as_view(), name='change_pass'),

    path('contact/', contact_view, name='contact'),
    path('instuctions/', InstructionsView.as_view(), name='Howtouse')
]

```

```

views.py
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView ,FormView
from django.contrib.auth.views import LoginView , PasswordResetView , PasswordChangeView
from django.contrib.messages.views import SuccessMessageMixin
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views import View
from django.views.generic import TemplateView
from django.urls import reverse_lazy
from django.contrib import messages
from django.contrib.auth import update_session_auth_hash
from django.shortcuts import redirect, render
from django.http import Http404
from base.models import *
from base.form import *
from django.core.mail import send_mail
from django.shortcuts import render
from django.http import HttpResponse

```

```

def classify_bmi(bmi):
    if bmi < 16:
        return "Severe Thinness"
    elif 16 <= bmi < 17:
        return "Moderate Thinness"
    elif 17 <= bmi < 18.5:
        return "Mild Thinness"
    elif 18.5 <= bmi < 25:
        return "Normal"
    elif 25 <= bmi < 30:
        return "Overweight"
    elif 30 <= bmi < 35:
        return "Obese Class I"
    elif 35 <= bmi < 40:
        return "Obese Class II"
    else:
        return "Obese Class III"

```

```

class UserRegisterview(CreateView):
    form_class = UserRegisterForms
    template_name = 'Authen/register.html'
    success_url = reverse_lazy('base:Login')

    def form_valid(self, form):
        user = form.save()
        messages.success(self.request, 'สมัครสมาชิกสำเร็จ! กรุณารอสักครู่...')
        return super().form_valid(form)

    def form_invalid(self, form):
        messages.error(self.request, 'เกิดข้อผิดพลาดในการลงทะเบียน กรุณาตรวจสอบข้อมูลอีกครั้ง.')
        return super().form_invalid(form)

class Userloginview(LoginView):
    template_name = 'Authen/login.html'

    def get_success_url(self):
        return reverse_lazy('base:home')

class HomepageView(LoginRequiredMixin, ListView):
    model = UserInfomation # โมเดลหลักที่ใช้สำหรับ ListView
    template_name = 'account.html'
    context_object_name = 'User_items'

    def get_queryset(self):

        return UserInfomation.objects.filter(id=self.request.user.id)

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        user = self.request.user
        personal_info = PersonalInformation.objects.filter(user=user).first()
        context['Personal_items'] = PersonalInformation.objects.filter(user=user)

        if personal_info and personal_info.height and personal_info.weight:
            height_m = personal_info.height / 100
            bmi = round(personal_info.weight / (height_m ** 2), 2)
            context['bmi'] = bmi
            context['bmi_category'] = classify_bmi(bmi)
        else:
            context['bmi'] = None
            context['bmi_category'] = "Not Available"

        return context

class ResetPasswordview(SuccessMessageMixin, PasswordResetView):
    template_name = r'Authen/password_reset.html'
    email_template_name = r'Authen/password_reset_email.html'
    subject_template_name = r'Authen/password_reset_subject.txt'
    success_message = 'เราได้ส่งลิงค์ในการ reset รหัสผ่านไปทางอีเมลที่คุณแจ้งแล้ว โปรดตรวจสอบที่emailของคุณ'
    success_url = reverse_lazy('base:Login')
    def form_valid(self, form):
        messages.success(self.request, self.success_message)
        return self.render_to_response(self.get_context_data(form=form))

class PersonalInformationUpdateView(LoginRequiredMixin, UpdateView):
    model = PersonalInformation
    fields = ['goal', 'job_name', 'job_type', 'job_hours', 'break_hours', 'age', 'height', 'weight', 'has_pain']
    template_name = 'update_user_data.html' # ใช้ Template เดียวกัน
    success_url = reverse_lazy('base:home')

    def get_object(self, queryset=None):
        obj, created = PersonalInformation.objects.get_or_create(user=self.request.user)
        return obj

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['form_type'] = 'personal_information' # ระบุประเภทฟอร์ม
        return context

class ChangeUsernameView(LoginRequiredMixin, SuccessMessageMixin, UpdateView):

```

```

model = UserInfomation
form_class = UserChangeForm
template_name = r'change/change_username.html'
success_url = reverse_lazy('base:home')
success_message = 'เปลี่ยนชื่อให้แล้วจะจิ้นจิ้นจิ้น'

def get_object(self):
    return self.request.user

class ChangePasswordView(LoginRequiredMixin, SuccessMessageMixin, PasswordChangeView):
    form_class = CustomPasswordChangeForm
    template_name = r'change/change_password.html'
    success_url = reverse_lazy('profile')
    success_message = "Password ถูกเปลี่ยนเรียบร้อยแล้ว!"

    def form_valid(self, form):
        response = super().form_valid(form)
        update_session_auth_hash(self.request, self.object)
        return response

def contact_view(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            name = form.cleaned_data['name']
            email = form.cleaned_data['email']
            message = form.cleaned_data['message']
            full_message = f"Message from {name} ({email}): \n\n{message}"

            # ส่งอีเมล
            send_mail(
                'Contact from User',
                full_message,
                email, # Email ผู้ส่ง
                ['sittotit.noreply@gmail.com'], # Email ทีมพัฒนา
            )
            return HttpResponse("Thank you for contacting us.")
        else:
            form = ContactForm()

    return render(request, 'contact.html', {'form': form})

class InstructionsView(TemplateView):
    template_name = 'Howto.html'
__init__.py
__init__.py
add_posture.py
from django.core.management.base import BaseCommand
from base.models import UserInfomation, PostureDetection , UserUsageHistory
from faker import Faker
import random
from datetime import datetime

fake = Faker()

class Command(BaseCommand):
    help = 'Add PostureDetection data for a specific user'

    def add_arguments(self, parser):
        parser.add_argument('username', type=str, help='Username of the user to add PostureDetection data for')
        parser.add_argument('count', type=int, help='Number of PostureDetection records to add')
        parser.add_argument('--timestamp', type=str, help='Timestamp for the records (format: YYYY-MM-DD HH:MM:SS)')

    def handle(self, *args, **kwargs):
        username = kwargs['username']
        count = kwargs['count']
        timestamp_str = kwargs.get('timestamp')

        try:
            user = UserInfomation.objects.get(username=username)
        except UserInfomation.DoesNotExist:
            self.stdout.write(self.style.ERROR(f'User with username "{username}" does not exist'))
            return

        if timestamp_str:

```

```

        try:
            custom_timestamp = datetime.strptime(timestamp_str, '%Y-%m-%d %H:%M:%S')
        except ValueError:
            self.stdout.write(self.style.ERROR(f'Invalid timestamp format. Use "YYYY-MM-DD HH:MM:SS".'))
            return
    else:
        custom_timestamp = None # Use auto-generated timestamps if not provided

    # Create PostureDetection records
    for _ in range(count):
        PostureDetection.objects.create(
            user=user,
            detection_time=fake.time_delta(),
            score=random.randint(50, 100),
            timestamp=custom_timestamp or fake.date_time_this_year(), # Use custom or random timestamp
        )

    self.stdout.write(self.style.SUCCESS(f'{count} PostureDetection records added for user "{username}"'))

```

clean_database.py

```

from django.core.management.base import BaseCommand
from base.models import PostureDetection, UserUsageHistory, UserInfomation

class Command(BaseCommand):
    help = 'Clear data from specific tables'

    def add_arguments(self, parser):
        parser.add_argument('--table', nargs='+', help='Specify table(s) to clear')

    def handle(self, *args, **options):
        tables = options['table']
        if 'PostureDetection' in tables:
            PostureDetection.objects.all().delete()
            self.stdout.write('PostureDetection cleared.')

        if 'UserUsageHistory' in tables:
            UserUsageHistory.objects.all().delete()
            self.stdout.write('UserUsageHistory cleared.')

        if 'UserInfomation' in tables:
            UserInfomation.objects.all().delete()
            self.stdout.write('UserInfomation cleared.')

        if not tables:
            self.stdout.write('No tables specified.')

```

random_users.py

```

from django.core.management.base import BaseCommand
from base.models import (
    UserInfomation, PersonalInformation, PersonalHealthInformation,
    UserUsageHistory, PostureDetection, NotificationLog
)
from faker import Faker
import random

fake = Faker()

class Command(BaseCommand):
    help = 'Generate sample data for testing'

    def add_arguments(self, parser):
        parser.add_argument('users', type=int, help='Number of users to create')
        parser.add_argument('--posture-count', type=int, default=1, help='Number of PostureDetection per user')

    def handle(self, *args, **kwargs):
        users_count = kwargs['users']
        posture_count = kwargs['posture_count']

        for _ in range(users_count):
            # สร้าง UserInfomation
            user = UserInfomation.objects.create_user(
                username=fake.user_name(),
                email=fake.email(),
                password='password123',
                contact_number=fake.phone_number(),
                role='Beginner Sitter',
            )

```

```

self.stdout.write(f'User {user.username} created')

# สร้าง PersonalInformation
PersonalInformation.objects.create(
    user=user,
    goal=random.choice(['Improve posture', 'Health maintenance', 'Posture correction']),
    job_name=fake.job(),
    job_type=fake.job(),
    job_hours=random.uniform(4, 10),
    break_hours=random.uniform(0.5, 2),
)
self.stdout.write(f'PersonalInformation for {user.username} created')

# สร้าง PersonalHealthInformation
PersonalHealthInformation.objects.create(
    user=user,
    age=random.randint(18, 65),
    height=random.uniform(150, 200),
    weight=random.uniform(50, 100),
    has_pain=random.choice([True, False]),
)
self.stdout.write(f'PersonalHealthInformation for {user.username} created')

# สร้าง UserUsageHistory
UserUsageHistory.objects.create(
    user=user,
    detect_type=random.choice(['Simple Detection', 'Advanced Detection']),
)

# สร้าง PostureDetection หลายรายการ
for _ in range(posture_count):
    PostureDetection.objects.create(
        user=user,
        detection_time=fake.time_delta(),
        score=random.randint(50, 100),
    )
self.stdout.write(f'{posture_count} PostureDetection records for {user.username} created')

# สร้าง NotificationLog
NotificationLog.objects.create(
    user=user,
    message=fake.sentence(),
    admin_message=fake.sentence(),
)
self.stdout.write(f'NotificationLog for {user.username} created')

self.stdout.write('Sample data generated successfully.')

```

__init__.py

0001_initial.py

Generated by Django 5.1.2 on 2024-10-18 13:52

```

import django.contrib.auth.models
import django.contrib.auth.validators
import django.db.models.deletion
import django.utils.timezone
from django.conf import settings
from django.db import migrations, models

```

```
class Migration(migrations.Migration):
```

```
    initial = True
```

```

    dependencies = [
        ("auth", "0012_alter_user_first_name_max_length"),
    ]

```

```

    operations = [
        migrations.CreateModel(
            name="UserInfomation",
            fields=[
                (
                    "id",
                    models.BigAutoField(
                        auto_created=True,

```

```

        primary_key=True,
        serialize=False,
        verbose_name="ID",
    ),
),
("password", models.CharField(max_length=128, verbose_name="password")),
(
    "last_login",
    models.DateTimeField(
        blank=True, null=True, verbose_name="last login"
    ),
),
(
    "is_superuser",
    models.BooleanField(
        default=False,
        help_text="Designates that this user has all permissions without explicitly assigning them.",
        verbose_name="superuser status",
    ),
),
(
    "username",
    models.CharField(
        error_messages={
            "unique": "A user with that username already exists."
        },
        help_text="Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.",
        max_length=150,
        unique=True,
        validators=[
            django.contrib.auth.validators.UnicodeUsernameValidator()
        ],
        verbose_name="username",
    ),
),
(
    "first_name",
    models.CharField(
        blank=True, max_length=150, verbose_name="first name"
    ),
),
(
    "last_name",
    models.CharField(
        blank=True, max_length=150, verbose_name="last name"
    ),
),
(
    "email",
    models.EmailField(
        blank=True, max_length=254, verbose_name="email address"
    ),
),
(
    "is_staff",
    models.BooleanField(
        default=False,
        help_text="Designates whether the user can log into this admin site.",
        verbose_name="staff status",
    ),
),
(
    "is_active",
    models.BooleanField(
        default=True,
        help_text="Designates whether this user should be treated as active. Unselect this instead of deleting accounts.",
        verbose_name="active",
    ),
),
(
    "date_joined",
    models.DateTimeField(
        default=django.utils.timezone.now, verbose_name="date joined"
    ),
),
),

```

```

("contact_number", models.CharField(blank=True, max_length=20)),
("role", models.CharField(max_length=50)),
("created_at", models.DateTimeField(auto_now_add=True)),
(
    "groups",
    models.ManyToManyField(
        blank=True,
        help_text="The groups this user belongs to. A user will get all permissions granted to each of their groups.",
        related_name="user_set",
        related_query_name="user",
        to="auth.group",
        verbose_name="groups",
    ),
),
(
    "user_permissions",
    models.ManyToManyField(
        blank=True,
        help_text="Specific permissions for this user.",
        related_name="user_set",
        related_query_name="user",
        to="auth.permission",
        verbose_name="user permissions",
    ),
),
],
options={
    "verbose_name": "user",
    "verbose_name_plural": "users",
    "abstract": False,
},
managers=[
    ("objects", django.contrib.auth.models.UserManager()),
],
),
migrations.CreateModel(
    name="NotificationLog",
    fields=[
        (
            "id",
            models.BigAutoField(
                auto_created=True,
                primary_key=True,
                serialize=False,
                verbose_name="ID",
            ),
        ),
        ("message", models.TextField()),
        ("notification_time", models.DateTimeField(auto_now_add=True)),
        ("admin_message", models.TextField(blank=True)),
        (
            "user",
            models.ForeignKey(
                on_delete=django.db.models.deletion.CASCADE,
                to=settings.AUTH_USER_MODEL,
            ),
        ),
    ],
),
),
migrations.CreateModel(
    name="PersonalHealthInformation",
    fields=[
        (
            "id",
            models.BigAutoField(
                auto_created=True,
                primary_key=True,
                serialize=False,
                verbose_name="ID",
            ),
        ),
        ("age", models.IntegerField()),
        ("height", models.FloatField()),
        ("weight", models.FloatField()),
        ("has_pain", models.BooleanField(default=False)),
    ],
),

```

```

        (
            "user",
            models.OneToOneField(
                on_delete=django.db.models.deletion.CASCADE,
                to=settings.AUTH_USER_MODEL,
            ),
        ),
    ],
),
migrations.CreateModel(
    name="PersonalInformation",
    fields=[
        (
            "id",
            models.BigAutoField(
                auto_created=True,
                primary_key=True,
                serialize=False,
                verbose_name="ID",
            ),
        ),
        (
            "goal",
            models.CharField(
                choices=[
                    ("Improve posture", "Improve posture"),
                    ("Health maintenance", "Health maintenance"),
                    ("Posture correction", "Posture correction"),
                ],
                max_length=50,
            ),
        ),
        ("job_name", models.CharField(max_length=100)),
        ("job_type", models.CharField(max_length=100)),
        ("job_hours", models.FloatField()),
        ("break_hours", models.FloatField()),
        (
            "user",
            models.OneToOneField(
                blank=True,
                null=True,
                on_delete=django.db.models.deletion.CASCADE,
                to=settings.AUTH_USER_MODEL,
            ),
        ),
    ],
),
migrations.CreateModel(
    name="PostureDetection",
    fields=[
        (
            "id",
            models.BigAutoField(
                auto_created=True,
                primary_key=True,
                serialize=False,
                verbose_name="ID",
            ),
        ),
        ("timestamp", models.DateTimeField(auto_now_add=True)),
        ("detection_time", models.DurationField()),
        ("score", models.IntegerField()),
        (
            "user",
            models.ForeignKey(
                on_delete=django.db.models.deletion.CASCADE,
                to=settings.AUTH_USER_MODEL,
            ),
        ),
    ],
),
migrations.CreateModel(
    name="UserUsageHistory",
    fields=[
        (

```

```

        "id",
        models.BigAutoField(
            auto_created=True,
            primary_key=True,
            serialize=False,
            verbose_name="ID",
        ),
    ),
    ("timestamp", models.DateTimeField(auto_now_add=True)),
    (
        "detect_type",
        models.CharField(
            choices=[
                ("Simple Detection", "Simple Detection"),
                ("Advanced Detection", "Advanced Detection"),
            ],
            max_length=50,
        ),
    ),
    ("usage_count", models.IntegerField()),
    (
        "user",
        models.ForeignKey(
            on_delete=django.db.models.deletion.CASCADE,
            to=settings.AUTH_USER_MODEL,
        ),
    ),
],
),
]

```

0002_alter_personalhealthinformation_age_and_more.py
 # Generated by Django 5.1.2 on 2024-11-08 04:52

```

import django.db.models.deletion
from django.conf import settings
from django.db import migrations, models

```

```

class Migration(migrations.Migration):

    dependencies = [
        ("base", "0001_initial"),
    ]

    operations = [
        migrations.AlterField(
            model_name="personalhealthinformation",
            name="age",
            field=models.IntegerField(blank=True, null=True),
        ),
        migrations.AlterField(
            model_name="personalhealthinformation",
            name="has_pain",
            field=models.BooleanField(blank=True, default=False, null=True),
        ),
        migrations.AlterField(
            model_name="personalhealthinformation",
            name="height",
            field=models.FloatField(blank=True, null=True),
        ),
        migrations.AlterField(
            model_name="personalhealthinformation",
            name="user",
            field=models.OneToOneField(
                blank=True,
                null=True,
                on_delete=django.db.models.deletion.CASCADE,
                to=settings.AUTH_USER_MODEL,
            ),
        ),
        migrations.AlterField(
            model_name="personalhealthinformation",
            name="weight",
            field=models.FloatField(blank=True, null=True),
        ),
    ]

```

```

migrations.AlterField(
    model_name="personalinformation",
    name="break_hours",
    field=models.FloatField(blank=True, null=True),
),
migrations.AlterField(
    model_name="personalinformation",
    name="goal",
    field=models.CharField(
        blank=True,
        choices=[
            ("Improve posture", "Improve posture"),
            ("Health maintenance", "Health maintenance"),
            ("Posture correction", "Posture correction"),
        ],
        max_length=50,
        null=True,
    ),
),
migrations.AlterField(
    model_name="personalinformation",
    name="job_hours",
    field=models.FloatField(blank=True, null=True),
),
migrations.AlterField(
    model_name="personalinformation",
    name="job_name",
    field=models.CharField(blank=True, max_length=100, null=True),
),
migrations.AlterField(
    model_name="personalinformation",
    name="job_type",
    field=models.CharField(blank=True, max_length=100, null=True),
),
]

```

0003_remove_userusagehistory_usage_count_and_more.py
 # Generated by Django 5.1.2 on 2024-11-26 01:41

```

import django.db.models.deletion
from django.db import migrations, models

```

```

class Migration(migrations.Migration):

    dependencies = [
        ("base", "0002_alter_personalhealthinformation_age_and_more"),
    ]

    operations = [
        migrations.RemoveField(
            model_name="userusagehistory",
            name="usage_count",
        ),
        migrations.RemoveField(
            model_name="userusagehistory",
            name="user",
        ),
        migrations.AddField(
            model_name="userusagehistory",
            name="posture_detection",
            field=models.ForeignKey(
                default=1,
                on_delete=django.db.models.deletion.CASCADE,
                to="base.posturedetection",
            ),
        ),
    ]

```

0004_alter_posturedetection_timestamp_and_more.py
 # Generated by Django 5.1.2 on 2024-11-27 12:10

```

from django.db import migrations, models

```

```

class Migration(migrations.Migration):

```

```

dependencies = [
    ("base", "0003_remove_userusagehistory_usage_count_and_more"),
]

operations = [
    migrations.AlterField(
        model_name="posturedetection",
        name="timestamp",
        field=models.DateTimeField(),
    ),
    migrations.AlterField(
        model_name="userusagehistory",
        name="timestamp",
        field=models.DateTimeField(),
    ),
]
0005_alter_posturedetection_timestamp_and_more.py
# Generated by Django 5.1.2 on 2024-11-27 12:12

```

```

from django.db import migrations, models

```

```

class Migration(migrations.Migration):

    dependencies = [
        ("base", "0004_alter_posturedetection_timestamp_and_more"),
    ]

    operations = [
        migrations.AlterField(
            model_name="posturedetection",
            name="timestamp",
            field=models.DateTimeField(auto_now_add=True),
        ),
        migrations.AlterField(
            model_name="userusagehistory",
            name="timestamp",
            field=models.DateTimeField(auto_now_add=True),
        ),
    ]

```

```

0006_remove_posturedetection_detection_time_and_more.py
# Generated by Django 5.1.2 on 2024-12-15 16:00

```

```

from django.db import migrations, models

```

```

class Migration(migrations.Migration):

    dependencies = [
        ("base", "0005_alter_posturedetection_timestamp_and_more"),
    ]

    operations = [
        migrations.RemoveField(
            model_name="posturedetection",
            name="detection_time",
        ),
        migrations.AddField(
            model_name="userusagehistory",
            name="detection_time",
            field=models.DateTimeField(blank=True, null=True),
        ),
        migrations.AlterField(
            model_name="userusagehistory",
            name="detect_type",
            field=models.CharField(
                choices=[
                    ("Photo Detection", "Photo Detection"),
                    ("Side-part Detection", "Side-part Detection"),
                ],
                max_length=50,
            ),
        ),
    ]

```

```

0007_remove_userusagehistory_timestamp.py

```

Generated by Django 5.1.2 on 2024-12-15 16:04

from django.db import migrations

class Migration(migrations.Migration):

```
dependencies = [
    ("base", "0006_remove_posturedetection_detection_time_and_more"),
]
```

```
operations = [
    migrations.RemoveField(
        model_name="userusagehistory",
        name="timestamp",
    ),
]
```

0008_posturedetection_detection_time_and_more.py

Generated by Django 5.1.2 on 2024-12-15 16:44

from django.db import migrations, models

class Migration(migrations.Migration):

```
dependencies = [
    ("base", "0007_remove_userusagehistory_timestamp"),
]
```

```
operations = [
    migrations.AddField(
        model_name="posturedetection",
        name="detection_time",
        field=models.DateTimeField(blank=True, null=True),
    ),
    migrations.AddField(
        model_name="userusagehistory",
        name="timestamp",
        field=models.DateTimeField(auto_now_add=True, null=True),
    ),
    migrations.AlterField(
        model_name="posturedetection",
        name="timestamp",
        field=models.DateTimeField(auto_now_add=True, null=True),
    ),
]
```

0009_alter_posturedetection_detection_time_and_more.py

Generated by Django 5.1.2 on 2024-12-16 17:46

from django.db import migrations, models

class Migration(migrations.Migration):

```
dependencies = [
    ("base", "0008_posturedetection_detection_time_and_more"),
]
```

```
operations = [
    migrations.AlterField(
        model_name="posturedetection",
        name="detection_time",
        field=models.FloatField(blank=True, null=True),
    ),
    migrations.AlterField(
        model_name="userusagehistory",
        name="detection_time",
        field=models.FloatField(blank=True, null=True),
    ),
]
```

0010_remove_posturedetection_detection_time_and_more.py

Generated by Django 5.1.2 on 2025-01-13 15:16

import django.db.models.deletion

from django.db import migrations, models

```

class Migration(migrations.Migration):

    dependencies = [
        ("base", "0009_alter_posturedetection_detection_time_and_more"),
    ]

    operations = [
        migrations.RemoveField(
            model_name="posturedetection",
            name="detection_time",
        ),
        migrations.RemoveField(
            model_name="userusagehistory",
            name="timestamp",
        ),
        migrations.AddField(
            model_name="personalinformation",
            name="age",
            field=models.IntegerField(blank=True, null=True),
        ),
        migrations.AddField(
            model_name="personalinformation",
            name="has_pain",
            field=models.BooleanField(blank=True, default=False, null=True),
        ),
        migrations.AddField(
            model_name="personalinformation",
            name="height",
            field=models.FloatField(blank=True, null=True),
        ),
        migrations.AddField(
            model_name="personalinformation",
            name="weight",
            field=models.FloatField(blank=True, null=True),
        ),
        migrations.AlterField(
            model_name="userusagehistory",
            name="posture_detection",
            field=models.ForeignKey(
                on_delete=django.db.models.deletion.CASCADE, to="base.posturedetection"
            ),
        ),
        migrations.DeleteModel(
            name="PersonalHealthInformation",
        ),
    ]
0011_alter_posturedetection_timestamp.py
# Generated by Django 5.1.2 on 2025-01-18 19:38

```

```

import django.utils.timezone
from django.db import migrations, models

```

```

class Migration(migrations.Migration):

    dependencies = [
        ("base", "0010_remove_posturedetection_detection_time_and_more"),
    ]

    operations = [
        migrations.AlterField(
            model_name="posturedetection",
            name="timestamp",
            field=models.DateTimeField(default=django.utils.timezone.now, null=True),
        ),
    ]
0012_alter_posturedetection_timestamp.py
# Generated by Django 5.1.2 on 2025-01-18 19:45

```

```

import datetime
from django.db import migrations, models

```

```
class Migration(migrations.Migration):

    dependencies = [
        ("base", "0011_alter_posturedetection_timestamp"),
    ]

    operations = [
        migrations.AlterField(
            model_name="posturedetection",
            name="timestamp",
            field=models.DateTimeField(
                default=datetime.datetime(
                    2025, 1, 18, 19, 45, 25, 683261, tzinfo=datetime.timezone.utc
                ),
                null=True,
            ),
        ),
    ]
0013_alter_posturedetection_timestamp.py
# Generated by Django 5.1.2 on 2025-01-18 19:56

import django.utils.timezone
from django.db import migrations, models
```

```
class Migration(migrations.Migration):

    dependencies = [
        ("base", "0012_alter_posturedetection_timestamp"),
    ]

    operations = [
        migrations.AlterField(
            model_name="posturedetection",
            name="timestamp",
            field=models.DateTimeField(default=django.utils.timezone.now, null=True),
        ),
    ]
__init__.py
account.html
{% extends 'base.html' %}

{% block main %}
```



ข้อมูลผู้ใช้งาน

{% for item in User_items %}

ชื่อผู้ใช้งาน: {{ item.username }}



รหัสผ่าน: *****



อีเมล: {{ item.email }}

ระดับ: {{ item.role }}

เบอร์โทร: {{ item.contact_number }}

{% endfor %}



ข้อมูลงาน

{% for item in Personal_items %}

ชื่องาน: {{ item.job_name }}

เวลาในการทำงาน: {{ item.job_hours|floatformat:"0" }} ชั่วโมง

เวลาในการพัก: {{ item.break_hours|floatformat:"0" }} ชั่วโมง

ประเภทงาน: {{ item.job_type }}

{% endfor %}

ข้อมูลสุขภาพ

{% for item in Personal_items %}

น้ำหนัก: {{ item.weight|floatformat:"2" }}

ส่วนสูง: {{ item.height|floatformat:"2" }}

ค่า BMI:

{% if bmi %}
 {{ bmi }}
{% else %}
 ไม่สามารถคำนวณได้
{% endif %}

สถานะ BMI:

{% if bmi_category %}
 {{ bmi_category }}
{% else %}
 ไม่สามารถคำนวณได้
{% endif %}

{% endfor %}

แก้ไขข้อมูลส่วนตัว

{% endblock %}
base.html
{% load static tailwind_tags %}

{% tailwind_css %}

SIT TO FIT

Dashboard

Instruction

Notification

Detection

Account

contact us

{% csrf_token %}
Logout

{% block main %}

{% endblock %}

base_dash.html
{% load static tailwind_tags %}

{% tailwind_css %}

SIT TO FIT

Dashboard

Instruction

Notification

Detection

Account

contact us



{% csrf_token %}
Logout

{% block main %}

{% endblock %}

```
{% load static tailwind_tags %}
```

```
{% tailwind_css %}
```

SIT TO FIT

Dashboard

Instruction

Notification

Detection

Account

contact us

{% csrf_token %}
Logout

{% block main %}

{% endblock %}

base_no_bar.html
{% load static tailwind_tags %}

{% tailwind_css %}

{% block content %}

{% endblock %}

contact.html
{% extends 'base.html' %}

{% block main %}



ติดต่อเรา

หากมีคำถามหรือข้อเสนอแนะ กรุณาส่งข้อความถึงเรา

{% csrf_token %}

ชื่อของคุณ

กรอกชื่อของคุณ

อีเมล

กรอกอีเมลของคุณ

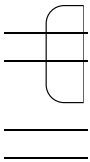
ข้อความ

พิมพ์ข้อความของคุณที่นี่

ส่งข้อความ

```
{% endblock %}
Howto.html
{% extends 'base.html' %}

{% block main %}
```



Introduction

ยินดีต้อนรับเข้าสู่เว็บไซต์ SIT TO FIT เว็บไซต์ตรวจจับท่านั่งเพื่อสุขภาพที่ยั่งยืน



Step-by-Step Instructions

SIT TO FIT วิธีการใช้งานง่ายๆด้วยหลัก 4S

1S

Select Camera

เลือกกล้องที่เหมาะสมสำหรับการตรวจจับท่านั่ง

2S

Setup Your body

จัดวางท่าทางร่างกายให้เหมาะสม โดยกสลองจะต้องจับผู้ใช้งานที่นั่งด้านข้าง และ ต้องเห็นสัดส่วนเต็มตัว

35

Start Posture Analysis

เริ่มทำการตรวจจับท่านี้

45

Stop and Check

หยุดทำการตรวจสอบจากนั้นดูผลคะแนนที่หน้า Dashboard

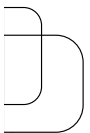
=====

=====

Frequently Asked Questions

▶ หากลืมรหัสผ่านต้องทำอย่างไร?

▶ ถ้าไม่มีกล่องจะสามารถตรวจสอบทานั่งได้ไหม?



```
{% endblock %}
update_user_data.html
{% extends 'base.html' %}

{% block main %}
```

อัปเดตข้อมูล

{% csrf_token %}

ข้อมูลส่วนตัว

กรุณากรอกข้อมูลเกี่ยวกับอาชีพและเป้าหมายของคุณ

{% for field in form %}

{{ field.label }}

```
        {{ field }}
        {% if field.help_text %}

        {{ field.help_text }}

        {% endif %}
        {% if field.errors %}

        {{ field.errors }}

        {% endif %}
```

```
{% endfor %}
```

บันทึกข้อมูล

ยกเลิก

```
{% endblock %}
login.html
{% extends "base_no_bar.html" %}

{% block content %}
```



Sit To Fit

นั่งให้ถูกท่า เพื่อสุขภาพที่ยั่งยืน

Sign in

{% csrf_token %}

Username

Enter Username

Password

Enter password

[Forgot Password?](#)

[Sign in](#)

[No Account? Sign up](#)

```
{% endblock %}
password_reset.html
{% load static tailwind_tags %}
{% block main %}
```

```
{% tailwind_css %}
```

```
{% if messages %}
```

```
{% endif %}
```

Forgot Password?

{% if form.errors %}

{% for key, value in form.errors.items %}

{{ value }}

{% endfor %}

{% endif %}

{% csrf_token %}

Email

Enter email

Submit

Create A New Account

Back To Login

{% endblock %}

password_reset_complete.html

{% extends "base_no_bar.html" %}

{% block main %}

Password Reset Successful

Your password has been set. You may go ahead and log in now.

[Login Here](#)

```
{% endblock %}
password_reset_confirm.html
{% extends "base_no_bar.html" %}

{% block main %}
```

Reset Your Password

```
{% if validlink %}
    {% if form.errors %}
```

Oops!

There were errors with your submission:

- {{ value }}

```
{% for key, value in form.errors.items %}

    {% endfor %}
```

```
{% endif %}
```

```
{% csrf_token %}
```

New Password

Enter your new password

Confirm New Password

Confirm your new password

Reset Password

{% else %}

Warning: The password reset link is invalid or has already been used. Please request a new password reset.

{% endif %}

{% endblock %}

password_reset_email.html

{% autoescape off %}

เพื่อเริ่มกระบวนการรีเซ็ตรหัสผ่านสำหรับบัญชี SIT TO FIT ของคุณ ({{ user.email }}),
กรุณาคlickลิงก์ด้านล่าง:

{{ protocol }}://{{ domain }}{% url 'base:password_reset_confirm' uidb64=uid token=token %}

หากการclickลิงก์ด้านบนไม่ได้ผล กรุณาคัดลอกและวาง URL ในหน้าต่างเบราว์เซอร์ใหม่แทน

{% endautoescape %}

register.html

{% extends "base_no_bar.html" %}

{% block content %}



Sit To Fit

นั่งให้ถูกท่า เพื่อสุขภาพที่ยั่งยืน

Welcome to SIT TO FIT

Sign in

{% csrf_token %}

Email:

Enter Email

{% if form.email.errors %}

{{ form.email.errors.as_text }}

{% endif %}

Username:

Enter Username

{% if form.username.errors %}

{{ form.username.errors.as_text }}

{% endif %}

Contact number:

Enter number

{% if form.contact_number.errors %}

{{ form.contact_number.errors.as_text }}

{% endif %}

Password:

Enter password

{% if form.password1.errors %}

{{ form.password1.errors.as_text }}

{% endif %}

Password confirmation:

Enter password again

{% if form.password2.errors %}

{{ form.password2.errors.as_text }}

{% endif %}

Sign up

Already have Account? | Sign in

{% if messages %}

{% endif %}

{% endblock %}
change_password.html
{% extends 'base_no_bar.html' %}

{% block content %}



เปลี่ยน Password

{% csrf_token %}

{{ form.as_p }}

บันทึก

```
{% endblock %}
change_username.html
{% extends 'base_no_bar.html' %}

{% block content %}
```



เปลี่ยน Username

{% csrf_token %}

Username

กรอกชื่อผู้ใช้งานใหม่



บันทึก

{% endblock %}